The Hunt: CTF Write-up

Category: Network forensics / ICS (Modbus/TCP)
Author of challenge: dexter
Difficulty: Medium, 150 pts (Q1 20 pts, Q2 30 pts, Q3 100 pts)
Artifact: `The_hunt.zip` containing `the_hunt.pcap`

> **Challenge text:** Our engineers noticed something wrong in the industrial pipeline, they have tasked you to investigate. Can you find what is lurking in the shadows?

Questions

1. What was the engineer IP?
2. What was the attacker IP?
3. What is the flag?

Answers

Question	Answer
Engineer IP	`172.28.0.30`
Attacker IP	`172.28.0.66`
Flag	`r00t{1_l0v3_m3_s0m3_m0dbus}`

> **Note on the terminal output below.** Blocks marked *captured output* are real output from the Python parser I ran while solving this. The `tshark` commands are the Kali equivalents of the same steps; run them in your own terminal to take screenshots. Their expected results match the captured output.

Step 1: Unpack and identify the artifact

```
┌──(enos㉿kali)-[~]
└─$ mkdir -p hunt && cd hunt

┌──(enos㉿kali)-[~/hunt]
└─$ unzip -o The_hunt.zip
Archive:  The_hunt.zip
  inflating: the_hunt.pcap

┌──(enos㉿kali)-[~/hunt]
└─$ ls -laR
.:
total 340
drwxr-xr-x 2 root root   4096 Oct 27 15:33 .
drwxr-xr-x 8 root root   4096 Oct 27 15:33 ..
-rw-r--r-- 1 root root 336584 Oct  3 06:36 the_hunt.pcap

┌──(enos㉿kali)-[~/hunt]
└─$ file *
the_hunt.pcap: pcap capture file, microsecond ts (little-endian) - version
2.4 (Ethernet, capture length 262144)
```

Captured output. A single classic pcap, about 330 KB, with 3,703 packets.

Step 2: Protocol and conversation overview

The first question for any pcap is what protocols are in it and who talks to whom.

```
┌──(enos㉿kali)-[~/hunt]
└─$ python3 parse.py
```

My parser gave the same picture. *Captured output:*

```
[((6, 502), 3703)]

('172.28.0.20', '172.28.0.10', 6, 502) 2218
('172.28.0.10', '172.28.0.20', 6, 46980) 1110
('172.28.0.21', '172.28.0.10', 6, 502) 224
('172.28.0.10', '172.28.0.21', 6, 36574) 113
('172.28.0.66', '172.28.0.10', 6, 502) 18
('172.28.0.10', '172.28.0.66', 6, 42230) 10
('172.28.0.30', '172.28.0.10', 6, 502) 6
('172.28.0.10', '172.28.0.30', 6, 44448) 4
```

Findings

- Every packet is TCP port 502, which is **Modbus/TCP**, an industrial control protocol. This matches the "industrial pipeline" wording in the challenge.
- `172.28.0.10` is the server (the PLC). Four clients talk to it: `.20`, `.21`, `.30` and `.66`.
- `.20` and `.21` send hundreds of packets, `.30` and `.66` very few. That already hints at "normal polling" versus "one-off activity".

Step 3: Decode Modbus function codes per client

In Wireshark or tshark the Modbus fields are `mbtcp.func_code`, `modbus.reference_num` and so on. Useful filters:

```
┌──(enos㉿kali)-[~/hunt]
└─$ python3 modbus.py
```

My parser's per-client summary. *Captured output:*

```
172.28.0.21 {(3, 1): 111}
172.28.0.20 {(3, 1): 554, (1, 1): 554}
172.28.0.30 {(3, 1): 1, (6, 1): 1}
172.28.0.66 {(3, 1): 1, (1, 1): 1, (5, 1): 2, (6, 1): 3, (16, 1): 1}
```

Modbus function codes seen: **1** = Read Coils, **3** = Read Holding Registers, **5** = Write Single Coil, **6** = Write Single Register, **16** = Write Multiple Registers.

Host	Behaviour	Role
`172.28.0.20`	554x FC3 and 554x FC1, read-only, steady polling	HMI / SCADA poller
`172.28.0.21`	111x FC3, read-only, steady polling	Historian / monitor
`172.28.0.30`	1 read, then 1 write	Engineer making a setpoint change
`172.28.0.66`	Read, then many writes, including FC16	**Attacker**

Step 4: Timeline of the two low-volume hosts

First-seen time per host. *Captured output:*

```
┌──(enos㉿kali)-[~/hunt]
└─$ python3 - <<'EOF'
import struct,socket,datetime
f=open('the_hunt.pcap','rb').read()
off=24;first={}
while off<len(f):
    a,b,cl,ol=struct.unpack('<IIII',f[off:off+16]);off+=16
    d=f[off:off+cl];off+=cl
    s=socket.inet_ntoa(d[26:30]); first.setdefault(s,a+b/1e6)
for s,t in sorted(first.items(),key=lambda x:x[1]):
print(s,datetime.datetime.utcfromtimestamp(t).strftime('%H:%M:%S'), round(t-
min(first.values()),1),'s')
EOF
172.28.0.21 06:27:44 0.0 s
172.28.0.10 06:27:44 0.0 s
172.28.0.20 06:27:44 0.2 s
172.28.0.30 06:29:14 90.1 s
172.28.0.66 06:30:14 150.3 s
```

The two pollers start with the capture. About 90 seconds in, `.30` appears, and about 60 seconds after that `.66` appears.

Step 5: Q1 and Q2, identifying the engineer and the attacker

The full requests from the two low-volume clients. *Captured output (frame, epoch time, source, transaction id, unit id, function code, payload hex):*

```
┌──(enos㉿kali)-[~/hunt]
└─$ python3 modbus.py
608 1791008954.55 172.28.0.30 tid 22078 uid 1 fc 3 000a0001
612 1791008954.55 172.28.0.30 tid 6505 uid 1 fc 6 000a026c
1014 1791009014.77 172.28.0.66 tid 21096 uid 1 fc 3 00000064
1018 1791009014.77 172.28.0.66 tid 5923 uid 1 fc 1 00000008
1219 1791009044.77 172.28.0.66 tid 41832 uid 1 fc 5 00020000
1321 1791009059.77 172.28.0.66 tid 18192 uid 1 fc 6 00040046
1720 1791009119.77 172.28.0.66 tid 57385 uid 1 fc 6 00040028
1723 1791009119.77 172.28.0.66 tid 30649 uid 1 fc 5 0002ff00
2221 1791009194.77 172.28.0.66 tid 40202 uid 1 fc 16
0014001c380384007200300030007400 7b0031005f006c00300076003300 5f006d0033005f007
30030006d0033005f006d00300064006200750073007d
3016 1791009314.77 172.28.0.66 tid 60268 uid 1 fc 6 0014028a
```

Reading the payloads

- Frame 608 and 612 (`.30`): read register 10, then write register 10 with `0x026c` (620). It is one quick read-then-adjust. This is how an engineer tunes a parameter, and nothing else follows.
- Frame 1014 onward (`.66`): reads 100 registers and 8 coils (reconnaissance), switches coil 2 off (`0x0000`) and later on (`0xff00`), sets register 4 to 70 and later 40, and ends with a large FC16 write and a final write to register 20. This is process manipulation by an unexpected host.

Q1 answer: **`172.28.0.30`** (note: inferred from behaviour, the capture does not label hosts).
Q2 answer: **`172.28.0.66`**

Step 6: Q3, the FC16 write carries the flag

Frame 2221 is a Write Multiple Registers request. The Modbus payload layout is:

```
0014          start address = 20
001c          quantity      = 28 registers
38            byte count    = 56
0384 0072 0030 0030 0074 007b 0031 005f 006c 0030 0076 0033 005f 006d 0033
005f
0073 0030 006d 0033 005f 006d 0030 0064 0062 0075 0073 007d
```

In Kali you can pull the same bytes straight out of the packet:

```
┌──(enos㉿kali)-[~/hunt]
└─$ python3 modbus.py | grep "fc 16"
2221 1791009194.77 172.28.0.66 tid 40202 uid 1 fc 16
0014001c380384007200300030074007b0031005f006c00300076003300 5f006d0033005f007
30030006d0033005f006d00300064006200750073007d
```

Each register is 16 bits, but all except the first have a high byte of `00`, so each one is a single ASCII character. The first register (`0x0384` = 900) is a plain numeric value, not a character. Decoding the remaining 27 registers. *Captured output:*

```
┌──(enos㉿kali)-[~/hunt]
└─$ python3 - <<'EOF'
h="0384 0072 0030 0030 0074 007b 0031 005f 006c 0030 0076 0033 005f 006d 0033
005f 0073 0030 006d 0033 005f 006d 0030 0064 0062 0075 0073 007d".split()
regs=[int(x,16) for x in h]
print(len(regs), regs[0], ''.join(chr(r) for r in regs[1:]))
EOF
28 900 r00t{1_l0v3_m3_s0m3_m0dbus}
```

```
regs = [int(x,16) for x in "0384 0072 0030 0030 0074 007b ... 007d".split()]
print(''.join(chr(r) for r in regs[1:]))
```

Flag: `r00t{1_l0v3_m3_s0m3_m0dbus}`

The attacker hid data inside what looks like a normal holding-register write. Frame 3016, a final write of 650 to register 20, then overwrote the first register, which is a typical cover-the-tracks step.

Key takeaways

1. **Start with the big picture.** Protocol hierarchy and conversation stats took one command and showed immediately that the whole capture is Modbus/TCP with one server and four clients.
2. **Behaviour beats labels.** ICS traffic is extremely regular. Hosts that only poll (`.20`, `.21`) are baseline; the interesting hosts are the ones that appear late, send few packets and issue **writes**.
3. **Know your Modbus function codes.** FC1/FC3 are reads; FC5/FC6/FC16 are writes. Any write from an unfamiliar host is suspect, and FC16 can carry arbitrary bulk data.
4. **Separate the engineer from the attacker by intent.** The engineer did a single read-then-write of one register. The attacker did reconnaissance, repeated coil and register manipulation, and a large multi-register write.
5. **Registers are 16-bit containers.** When values like `0072 0030 0030 0074` appear, check whether the high bytes are zero and decode as ASCII. The first register was a value, not text, which is why decoding from index 1 gave a clean flag.
6. **No tshark? No problem.** A pcap is a simple format; a few lines of Python with `struct` were enough to parse every packet, which is a useful fallback on a locked-down box.
7. **Caveat on Q1.** The engineer IP is a judgement call from behaviour. If a platform rejects it, check `.20` and `.21` next.

Tools used

`unzip`, `file`, `tshark` / Wireshark (Modbus dissector), Python 3 (`struct`, `socket`)